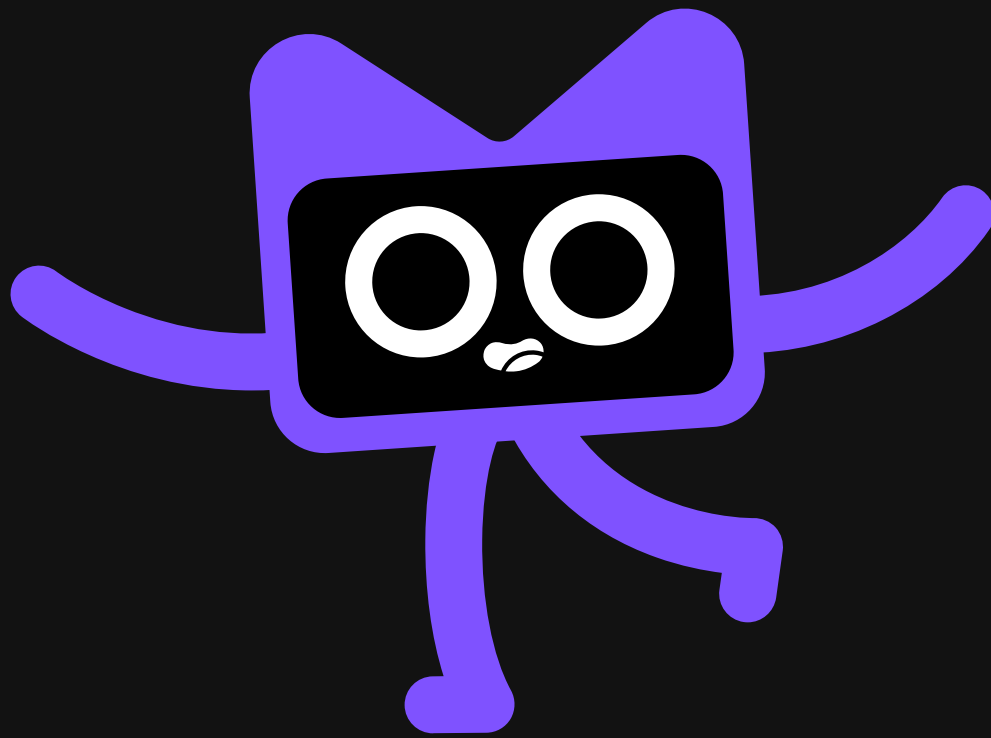


Why Spring Boot feels smaller in Kotlin

Frederik Pietzko



Why does Spring feel smaller in Kotlin?



Why does Spring feel smaller in Kotlin?

Kotlin features



Why does Spring feel smaller in Kotlin?

Kotlin features

- concise Syntax compare to Java
- Strict Nullability
- Smart Casts
- Extension Functions
- Default Arguments



Why does Spring feel smaller in Kotlin?

Kotlin features

for Spring

- concise Syntax compare to Java
- Strict Nullability
- Smart Casts
- Extension Functions
- Default Arguments



Why does Spring feel smaller in Kotlin?

Kotlin features

- concise Syntax compare to Java
- Strict Nullability
- Smart Casts
- Extension Functions
- Default Arguments

for Spring

- Kotlin Extensions for Spring
- Kotlin DSLs in Spring
- JSpecify
- Compiler Plugins
- Kotlin Ecosystem



Concise Syntax

```
public Pet getPet(Long id) {  
    return repository.findById(id)  
        .orElseThrow(() → new IllegalArgumentException("Pet not found"));  
}
```

```
@PetService  
class PetServiceImpl implements PetService {  
    private final PetRepository repository;  
    private final PetValidator validator;  
  
    public PetService(PetRepository repository, PetValidator validator) {  
        this.repository = repository;  
        this.validator = validator;  
    }  
}
```

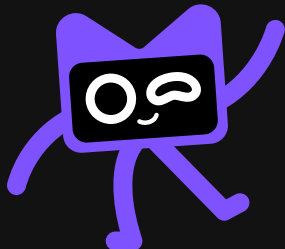


Concise Syntax

```
fun getPet(id: Long) = requireNotNull(repository.findByIdOrNull(id)) { "Pet not found" }
```

```
@PetService
class PetServiceImpl implements PetService {
    private final PetRepository repository;
    private final PetValidator validator;

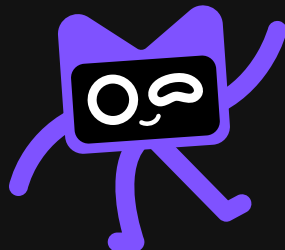
    public PetService(PetRepository repository, PetValidator validator) {
        this.repository = repository;
        this.validator = validator;
    }
}
```



Concise Syntax

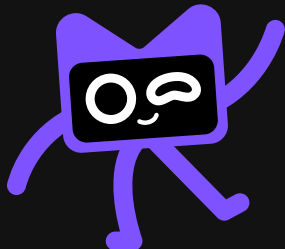
```
fun getPet(id: Long) = requireNotNull(repository.findByIdOrNull(id)) { "Pet not found" }
```

```
@PetService  
class PetServiceImpl(  
    private val repository: PetRepository,  
    private val validator: PetValidator,  
) : PetService
```



Strict Nullability

```
public Visit scheduleVisit(Long petId, OffsetDateTime at) {  
    final var pet = petRepository.findById(petId).orElseNull();  
    final var petName = pet.getName(); // potentially throws NullPointerException  
    return visitRepository.save(new Visit(petName, at))  
}
```



Strict Nullability

```
fun scheduleVisit(petId: Long, at: OffsetDateTime): Visit {  
    val pet = petRepository.findByIdOrNull(petId) // Type: Pet?  
    val petName = pet.name // compiler error  
    return visitRepository.save(Visit(petName, at))  
}
```



Strict Nullability

```
fun scheduleVisit(petId: Long, at: OffsetDateTime): Visit {  
    val pet = petRepository.findByIdOrNull(petId) // Type: Pet?  
    if (pet == null) throw IllegalArgumentException("Pet with id: $petId not found!")  
    val petName = pet.name // Pet is smart cast to Pet  
    return visitRepository.save(Visit(petName, at))  
}
```



Strict Nullability

```
fun scheduleVisit(petId: Long, at: OffsetDateTime): Visit {  
    val pet = petRepository.findByIdOrNull(petId) ?: throw IllegalArgumentException("Pet with id: $petId not found!")  
    val petName = pet.name // Pet is smart cast to Pet  
    return visitRepository.save(Visit(petName, at))  
}
```



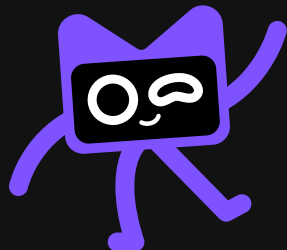
Strict Nullability

```
fun scheduleVisit(petId: Long, at: OffsetDateTime): Visit {  
    val pet = petRepository.findByIdOrNull(petId) // Type: Pet?  
    requireNotNull(pet) { "Pet with id: $petId not found!" }  
    val petName = pet.name // Pet is smart cast to Pet  
    return visitRepository.save(Visit(petName, at))  
}
```



Strict Nullability

```
fun scheduleVisit(petId: Long, at: OffsetDateTime): Visit {  
    val pet = requireNotNull(petRepository.findByIdOrNull(petId)) {  
        "Pet with id: $petId not found!"  
    }  
    val petName = pet.name // Pet is smart cast to Pet  
    return visitRepository.save(Visit(petName, at))  
}
```



Strict Nullability

```
inline fun <T : Any> requireNotNull(value: T?, lazyMessage: () → Any): T {  
    contract {  
        returns() implies (value ≠ null)  
    }  
  
    if (value == null) {  
        throw IllegalArgumentException(lazyMessage.toString())  
    } else {  
        return value  
    }  
}
```



Smart Casts

```
val pet = petRepository.findByIdOrNull(petId)
if (pet == null) {
    // pet has type Pet?
} else {
    // pet has type Pet!
}
```



Smart Casts

```
val stringOrInt: Any = if (Random.nextBoolean()) "string" else 1
if (stringOrInt is String) {
    // stringOrInt is smart cast to String
}
if(stringOrInt is Int) {
    // stringOrInt is smart cast to Int
}
// stingOrInt is Any
```



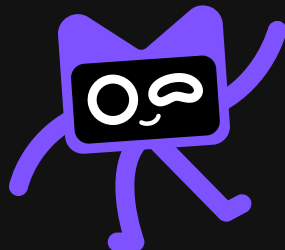
Smart Casts

```
val stringOrInt: Any = if (Random.nextBoolean()) "string" else 1
when(stringOrInt) {
    is String → // stringOrInt is smart cast to String
    is Int → // stringOrInt is smart cast to Int
    else → // stringOrInt is Any
}
```



Smart Casts

```
sealed interface SchedulingResult {  
    data class Scheduled(val visit: Visit): SchedulingResult  
    data class SlotTaken(val takenBy: Visit): SchedulingResult  
    data object VetOnVacation: SchedulingResult  
}  
  
fun scheduleVisit(petId: Long, at: OffsetDateTime): SchedulingResult  
  
return when(val result = scheduleVisit(...)) {  
    is Scheduled → result.visit  
    is SlotTaken → error("Slot unavailable, taken by ${result.takenBy}")  
    VetOnVacation → error("Vet on vacation")  
}
```



Smart Casts

```
sealed interface SchedulingResult {  
    data class Scheduled(val visit: Visit): SchedulingResult  
    data class SlotTaken(val takenBy: Visit): SchedulingResult  
    data object VetOnVacation: SchedulingResult  
    data object VetSick: SchedulingResult // newly added  
}  
  
fun scheduleVisit(petId: Long, at: OffsetDateTime): SchedulingResult  
  
// compiler error!  
return when(val result = scheduleVisit(...)) {  
    is Scheduled → result.visit  
    is SlotTaken → error("Slot unavailable, taken by ${result.takenBy}")  
    VetOnVacation → error("Vet on vacation")  
}
```



Smart Casts

```
sealed interface SchedulingResult {  
    data class Scheduled(val visit: Visit): SchedulingResult  
    data class SlotTaken(val takenBy: Visit): SchedulingResult  
    data object VetOnVacation: SchedulingResult  
    data object VetSick: SchedulingResult // newly added  
}  
  
fun scheduleVisit(petId: Long, at: OffsetDateTime): SchedulingResult  
  
val result = scheduleVisit(...)  
  
if(result is Scheduled) {  
    return result.visit  
}  
  
// if we enable -Xdata-flow-based-exhaustiveness  
return when(result) {  
    // no longer needed to check Scheduled  
    is SlotTaken → error("Slot unavailable, taken by ${result.takenBy}")  
    VetOnVacation → error("Vet on vacation")  
    VetSick → error("Vet is sick")  
}
```



Extension Functions

```
val pet = repository.findByIdOrNull(petId)
```



Extension Functions

```
fun <T: Any, ID: Any> CrudRepository<T, ID>.findByIdOrNull(id: ID): T? = findById(id).orElse(null)
```



Extension Functions

```
@Entity
class Pet(
    @Id
    var id: Long? = null,
    var name: String
) {
    fun toDto() = PetDto(id, name)
}
```



Extension Functions

```
package org.example.domain

@Entity
class Pet(
    @Id
    var id: Long? = null,
    var name: String
)

package org.example.api

class PetController {
    @GetMapping
    fun getPet(id: Long): PetDto {
        val pet = requireNotNull(repository.findByIdOrNull(id))
        return pet.toDto()
    }

    private fun Pet.toDto() = PetDto(id, name)
}
```



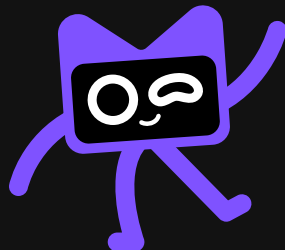
Default Arguments

```
fun scheduleVisit(petId: Long, at: OffsetDateTime = OffsetDateTime.now() + Duration.ofDays(2)) : Visit {  
    // ...  
}
```



Default Arguments

```
@JvmOverloads  
fun scheduleVisit(petId: Long, at: OffsetDateTime = OffsetDateTime.now() + Duration.ofDays(2)) : Visit {  
    // ...  
}
```



Default Arguments

```
public Visit scheduleVisit(Long petId, OffsetDateTime at) {  
    // ...  
}  
  
public Visit scheduleVisit(Long petId) {  
    return scheduleVisit(petId, OffsetDateTime.now().plus(Duration.ofDays(2)))  
}
```



Kotlin specific Extension Functions in Spring

```
val pet = petRepository.findByIdOrNull(id)
```



Kotlin specific Extension Functions in Spring

```
val pet = petRepository.findByIdOrNull(id)  
  
val pet = restTemplate.getForEntity<Pet>("/")
```



Kotlin specific Extension Functions in Spring

```
val pet = petRepository.findByIdOrNull(id)

val pet = restTemplate.getForEntity<Pet>("/")

val pet = jdbcTemplate.queryForObject<Pet>(...)
```



Kotlin specific Extension Functions in Spring

```
inline fun <reified T: Any> RestOperations.getForEntity(url: String, vararg uriVariables: Any?): ResponseEntity<T> =  
    getForEntity(url, T::class.java, *uriVariables)
```



JSpecify

- Standard for nullability Annotations
- by Google, JetBrains, Meta, Microsoft, Oracle, Broadcom, ...
- Supported by IntelliJ IDEA, NullAway, Checker Framework
- and Kotlin!
- Spring Framework 7 & Spring Boot 4 completely annotated ⇒ seamless Kotlin interop

Kotlin DSLs in Spring

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) {
    return http
        .cors(cors → cors.disable())
        .csrf(csrf → csrf.disable())
        .authorizeHttpRequests(auth → auth
            .requestMatchers("/admin/**").hasRole("ADMIN")
            .anyRequest().permitAll())
        .formLogin(Customizer.withDefaults())
        .httpBasic(Customizer.withDefaults())
        .build();
}
```



Kotlin DSLs in Spring

```
@Bean
fun filterChain(http: HttpSecurity): SecurityFilterChain {
    http {
        cors { disable() }
        csrf { disable() }
        authorizeHttpRequests {
            authorize("/admin/**", hasRole("ADMIN"))
            authorize(anyRequest, authenticated)
        }
        formLogin { }
        httpBasic { }
    }
    return http.build()
}
```



Kotlin DSLs in Spring

```
@Test
public void testSomething() throws Exception {
    mockMvc.perform(
        MockMvcRequestBuilders.get("/pet/{id}", 1)
            .accept(MediaType.APPLICATION_JSON)
    )
    .andExpect(MockMvcResultMatchers.status().isOk())
    .andExpect(MockMvcResultMatchers.jsonPath("$.id").value(1));
}
```



Kotlin DSLs in Spring

```
@Test
fun `test something`() {
    mockMvc.get("/pet/{id}", 1) {
        accept = APPLICATION_JSON
    }.andExpect {
        status { isOk() }
        jsonPath("$.id") {
            value(1)
        }
    }
}
```



Kotlin DSLs in Spring

```
class Registrar implements BeanRegistrar {  
  
    @Override  
    public void register(BeanRegistry registry, Environment env) {  
        registry.registerBean(PetService.class);  
  
        if (env.matchesProfiles("dev")) {  
            registry.registerBean(TestingController.class);  
        }  
    }  
}  
  
@Configuration  
@Import(Registrar.class)  
class Config {}
```



Kotlin DSLs in Spring

```
class Registrar : BeanRegistrarDsl({  
    registerBean<PetService>()  
    if (env.matchesProfiles("dev")) {  
        registerBean<TestingController>()  
    }  
})
```

```
@Configuration  
@Import(Registrar::class)  
class Config
```



Kotlin DSLs in Spring

```
class Registrar implements BeanRegistrar {  
  
    @Override  
    public void register(BeanRegistry registry, Environment env) {  
        registry.registerBean(PetService.class);  
  
        if (env.matchesProfiles("dev")) {  
            registry.registerBean(TestingController.class,  
                (c) → c.supplier(context →  
                    new TestingController(context.bean(PetService.class))  
                )  
            );  
        }  
    }  
}  
  
@Configuration  
@Import(Registrar.class)  
class Config {}
```



Kotlin DSLs in Spring

```
class Registrar : BeanRegistrarDsl({  
    registerBean<PetService>()  
    if (env.matchesProfiles("dev")) {  
        registerBean { TestingController(bean()) }  
    }  
})  
  
@Configuration  
@Import(Registrar::class)  
class Config
```



Kotlin DSLs in Spring

```
registerBean {  
    val http = bean<HttpSecurity>()  
    http {  
        cors { disable() }  
        csrf { disable() }  
        authorizeHttpRequests {  
            authorize("/admin/**", hasRole("ADMIN"))  
            authorize(anyRequest, authenticated)  
        }  
        formLogin { }  
        httpBasic { }  
    }  
    http.build()  
}
```



Router DSL

```
fun petHandler(petService: PetService) = router {  
    "/pets".nest {  
        GET {  
            val pets = petService.getAllPets()  
            ok().body(pets)  
        }  
        POST {  
            val pet = petService.createPet(it.body<PetDto>())  
            ok().body(pet)  
        }  
       ("/{id}").nest {  
            GET {  
                val pet = petService.getById(it.pathVariable("id").toLong())  
                ok().body(pet)  
            }  
            DELETE {  
                petService.delete(it.pathVariable("id").toLong())  
                noContent().build()  
            }  
        }  
    }  
}
```



Router DSL

```
class Registrar : BeanRegistrarDsl({  
    registerBean<PetService>()  
    registerBean { petHandler(bean()) }  
})
```

```
@Configuration  
@EnableAutoConfiguration  
@Import(Registrar::class)  
class Application
```



When should you use these DSLs?

- Small to medium microservices
- startup time important
- you like functional style programming & apis
- always use test DSLs



What makes Kotlin & Spring interop comfortable?



What makes Kotlin & Spring interop comfortable?

final



What makes Kotlin & Spring interop comfortable?

final

Classes, methods and properties are final by default!



What makes Kotlin & Spring interop comfortable?

final



Classes, methods and
properties are final by
default!



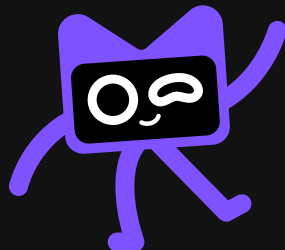
What makes Kotlin & Spring interop comfortable?

final



```
kotlin("plugin.spring")
```

Classes, methods and
properties are final by
default!



Spring Compiler Plugin

```
@Service
open class PetService {
    @Transactional
    open fun scheduleVisit(petId: Long, at: OffsetDateTime = OffsetDateTime.now() + Duration.ofDays(2)): Visit {
        ...
    }
}
```



Spring Compiler Plugin

```
@Service
class PetService {
    @Transactional
    fun scheduleVisit(petId: Long, at: OffsetDateTime = OffsetDateTime.now() + Duration.ofDays(2)): Visit {
        ...
    }
}
```



JPA Compiler Plugin

```
plugin {  
    kotlin("plugin.jpa")  
}
```

```
@Entity  
open class Pet() {  
    @Id  
    open var id: Long? = null  
    open var name: String? = null  
  
    constructor(  
        id: Long? = null,  
        name: String? = null,  
    ) : this() {  
        this.id = id  
        this.name = name  
    }  
}
```



JPA Compiler Plugin

```
plugin {  
    kotlin("plugin.jpa")  
}
```

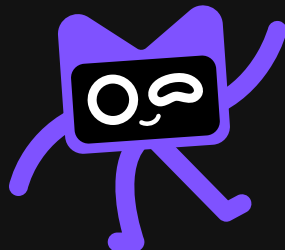
```
@Entity  
class Pet(  
    @Id  
    var id: Long? = null,  
    var name: String,  
)
```



Lombok?

- Kotlin cannot see Lombok's generated declarations
- Historically presented a blocker to Kotlin Adoption
- experimental Kotlin Lombok Plugin in 1.5.20
- promoted to alpha in 2.3.20
- runs lombok before `kotlinc`

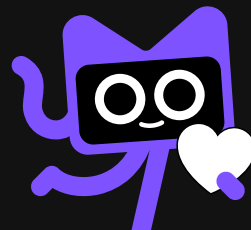
```
plugins {  
    kotlin("plugin.lombok")  
}
```



Future Improvements for Kotlin Lombok Plugin

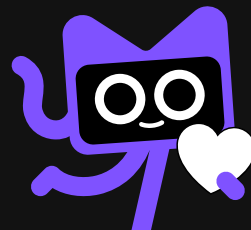
```
@Builder
@AllArgsConstrcutor
@Setter
@Getter
class Pet {
    private Long id;
    private String name;
    private Breed race;

    enum Breed {
        CAT, DOG, HORSE,
    }
}
```



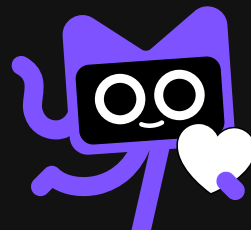
Future Improvements for Kotlin Lombok Plugin

```
class Pet(  
    var id: Long,  
    var name: String,  
    var race: Breed,  
) {  
    enum class Breed {  
        CAT, DOG, HORSE,  
    }  
}
```

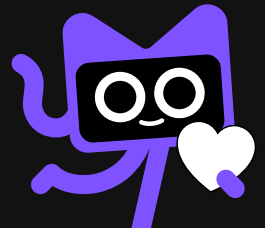


Future Improvements for Kotlin Lombok Plugin

```
@Builder
class Pet(
    var id: Long,
    var name: String,
    var race: Breed,
) {
    enum class Breed {
        CAT, DOG, HORSE,
    }
}
```

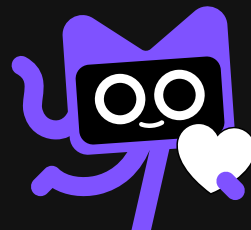


Future Improvements for Kotlin Lombok Plugin



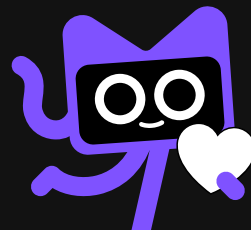
Future Improvements for Kotlin Lombok Plugin

- `@Builder`



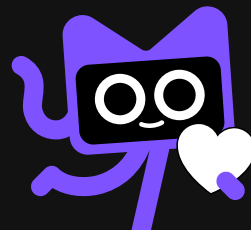
Future Improvements for Kotlin Lombok Plugin

- `@Builder`
- `@Log` , `@Slf4j` , ...



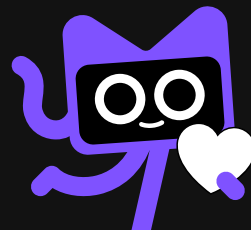
Future Improvements for Kotlin Lombok Plugin

- `@Builder`
- `@Log` , `@Slf4j` , ...
- `@NoArgsConstructor`



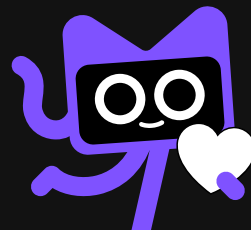
Future Improvements for Kotlin Lombok Plugin

- `@Builder`
- `@Log` , `@Slf4j` , ...
- `@NoArgsConstructor`
- `@EqualsAndHashCode`



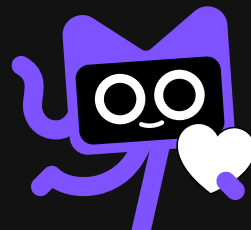
Future Improvements for Kotlin Lombok Plugin

- `@Builder`
- `@Log` , `@Slf4j` , ...
- `@NoArgsConstructor`
- `@EqualsAndHashCode`
- `@ToString`



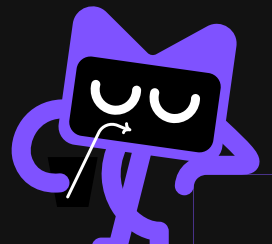
Future Improvements for Kotlin Lombok Plugin

- `@Builder`
- `@Log` , `@Slf4j` , ...
- `@NoArgsConstructor`
- `@EqualsAndHashCode`
- `@ToString`
- will probably arrive in `2.5.x`



Kotlin Ecosystem

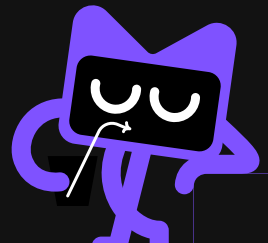
- Mappie
- MockK
- Kotest
- Exposed



Mappie

- alternative to MapStruct
- Compiler Plugin
- very good error messages

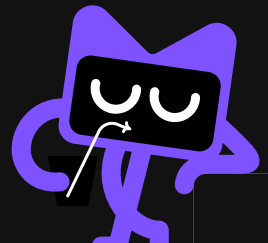
```
object PetMapper : ObjectMappie<Pet, PetDto>
```



Mappie

- alternative to MapStruct
- Compiler Plugin
- very good error messages

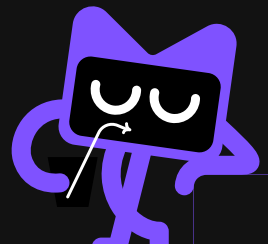
```
Target PetDto::id automatically resolved from Pet::id but cannot assign source type Long? to target type Long
```



Mappie

- alternative to MapStruct
- Compiler Plugin
- very good error messages

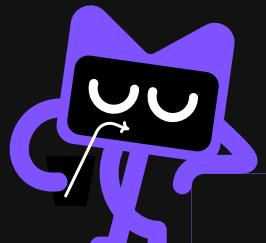
```
object PetMapper : ObjectMappie<Pet, PetDto> {  
    override fun map(from: Pet): PetDto = mapping {  
        PetDto::id fromPropertyNotNull from::id  
    }  
}
```



MockK

- excellent mocking library for Kotlin
- Kotlin DSL
- Spring Support through `SpringMockK`

```
val pet = mockk<Pet>()
every { pet.name } returns "Fido"
```

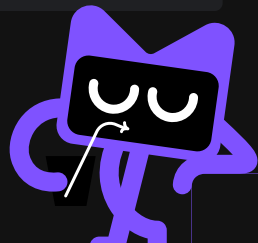


MockK

- excellent mocking library for Kotlin
- Kotlin DSL
- Spring Support through `SpringMockK`

```
@SpringBootTest
class PetControllerTest {
    @MockkBean
    private lateinit var petService: PetService

    @Test
    fun `should return pet`() {
        every { petService.getPet(1L) } returns Pet(1L, "Fido")
        // ...
    }
}
```



Kotest

- Test Framework with multiple styles
- assertions
- property testing

```
class MyTests : FunSpec({
    test("String length should return the length of the string") {
        "sammy".length shouldBe 5
        "".length shouldBe 0
    }
})

`val age = integer("age")
`val breed = varchar("breed", 50)`kotlin

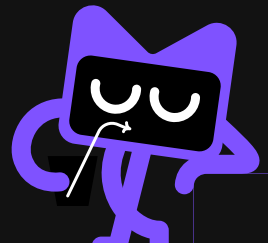
class MyTests : ShouldSpec({
    should("return the length of the string") {
        "sammy".length shouldBe 5
        "".length shouldBe 0
    }
})
```



Kotest

- Test Framework with multiple styles
- assertions
- property testing

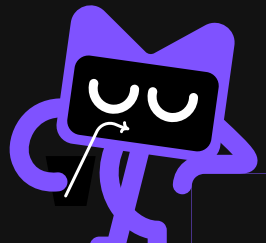
```
class MyTests : BehaviourSpec({  
    given("a string") {  
        `when`("i measure it's length") {  
            then("it should have length of 5") {  
                "sammy".length shouldBe 5  
            }  
        }  
    }  
})
```



Kotest

- Test Framework with multiple styles
- assertions
- property testing

```
@Test
fun `length of string`() {
    "sammy".length shouldBe 5
}
```



Exposed by JetBrains

- Kotlin DB library with ORM & query DSL flavours
- JDBC & R2DBC Support
- Generate Flyway migrations from Table definitions
- Spring Boot integration

```
object PetTable : LongIdTable() {  
    val name = varchar("name", 50)  
    val age = integer("age")  
    val breed = enumeration<Breed>("breed")  
}  
  
data class Pet(  
    val id: Long,  
    val name: String,  
    val age: Int,  
    val breed: Breed,  
)
```



Exposed by JetBrains

- Kotlin DB library with ORM & query DSL flavours
- JDBC & R2DBC Support
- Generate Flyway migrations from Table definitions
- Spring Boot integration

```
@Transactional
fun getPetById(id: Long) = PetTable.selectAll().where {
    PetTable.id eq id
}.firstOrNull()?.toPet()

fun ResultRow().toPet() = Pet(
    id = this[PetTable.id],
    name = this[PetTable.name],
    age = this[PetTable.age],
    breed = this[PetTable.breed],
)
```



Exposed by JetBrains

- Kotlin DB library with ORM & query DSL flavours
- JDBC & R2DBC Support
- Generate Flyway migrations from Table definitions
- Spring Boot integration

```
class PetEntity(id: EntityID<Long>) : LongEntity(id) {  
    companion object : LongEntityClass<PetEntity>(PetTable)  
    val name by PetTable.name  
    val age by PetTable.age  
    val breed by PetTable.breed  
}  
  
fun createPet() {  
    PetEntity.new {  
        name = "Loki"  
        age = 10  
        breed = Breed.CAT  
    }  
}  
  
fun findPetById(petId: Long): PetEntity = PetEntity.findById(petId)
```



Q & A

Slides

